

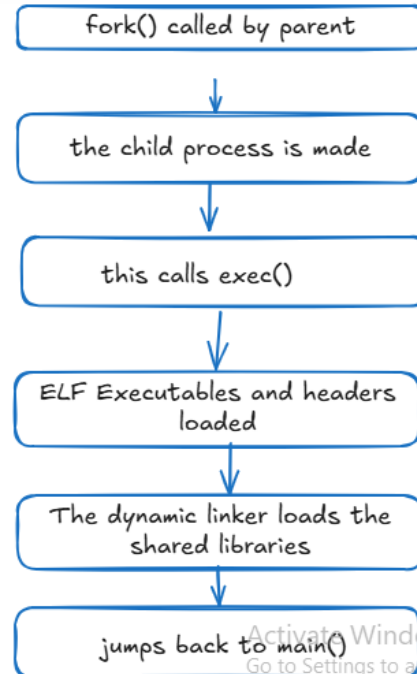
Q2-REPORT

Linux:

To move canvas, hold mouse wheel or spacebar while dragging, or use the hand tool

Linux

A newly created process in Linux usually starts a `fork()` system call, which spawns a copy of its parent as a child. The child then runs an `exec()`, which swaps out its memory space for that of a new program. The ELF (Executable and Linkable Format) file, headers, and sections are brought into memory at this point. The Linux loader then lays out all necessary shared libraries and works out their dependencies. The control then transfers to the entry point of programs, usually a `main` function. The two-stage method ensures reliability and separation when constructing a process.



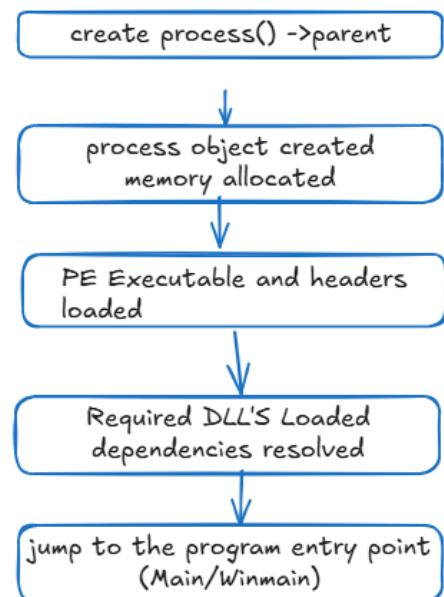
Windows:



To move canvas, hold mouse wheel or spacebar while dragging, or use the hand tool

windows

On Windows, initialization begins when the `CreateProcess()` system call is issued. The call establishes a new process and its initial thread concurrently. The operating system initializes the process's virtual address space and loads the appropriate parts of the executable into memory. Then, the PE loader takes care of solving imports, linking dynamic libraries, and initializing system structures. Then, initialization of a process's environment, such as command-line arguments and handles, is performed. Control then passes to a program's entry point, where initialization begins in earnest.



Q3-Execution Flow

```
int
main(void)
{
    static char buf[100];
    int fd;

    // Ensure that three file descriptors are open.
    while((fd = open("console", O_RDWR)) >= 0){
        if(fd >= 3){
            close(fd);
            break;
        }
    }
}
```

As the main function begins, the buffer is allocated space of buff[100] to run commands. The fd=open ensures that the three file descriptors are open(stdin,stdout and stderr) and mapped to the console.

```
// Read and run input commands.
while(getcmd(buf, sizeof(buf)) >= 0){
    char *cmd = buf;
    while (*cmd == ' ' || *cmd == '\t')
        cmd++;
    if (*cmd == '\n') // is a blank command
        continue;
    if(cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' '){
        // Chdir must be called by the parent, not the child.
        cmd[strlen(cmd)-1] = 0; // chop \n
        if(chdir(cmd+3) < 0)
            fprintf(2, "cannot cd %s\n", cmd+3);
    } else {
        if(fork1() == 0)
            runcmd(parsecmd(cmd));
        wait(0);
    }
}
```

Then, it uses `getcnd()` to read the user inputs from buffer. Checks are done to make sure input is not blank. Separate cases for “cd” are considered.

In the else section, `fork()` is used to create a child and run the command through `getcnd()`.

```
int
getcnd(char *buf, int nbuf)
{
    write(2, "$ ", 2);
    memset(buf, 0, nbuf);
    gets(buf, nbuf);
    if(buf[0] == 0) // EOF
        return -1;
    return 0;
}
```

`Getcmd()` is the function that prints the \$ sign on the console using `fd 2`.

Coming back to the else statement, it has `runcmd()` which does several operations using switch cases (EXEC, REDIR, PIPE, LIST, BACK):

```
void
runcmd(struct cmd *cmd)
{
    int p[2];
    struct backcmd *bcmd;
    struct execcmd *ecmd;
    struct listcmd *lcmd;
    struct pipecmd *pcmd;
    struct redircmd *rcmd;

    if(cmd == 0)
        exit(1);

    switch(cmd->type){
    default:
        panic("runcmd");

    case EXEC:
        ecmd = (struct execcmd*)cmd;
        if(ecmd->argv[0] == 0)
            exit(1);
        exec(ecmd->argv[0], ecmd->argv);
        fprintf(2, "exec %s failed\n", ecmd->argv[0]);
        break;

    case REDIR:
```

In this scenario,run `cmdcall exec()` to execute the program as it can be seen in the screenshot attached above. Hence,the file is searched for and executed. Since it is a system call,kernel is used.